EXPERIMENT 7

D15A-57

Aim: Build an Artificial Neural Network (ANN) using Keras/TensorFlow

Theory:

1. Dataset Source
Custom dataset : Link

2. Dataset Description

The dataset contains time-series fitness activity records collected from wearable fitness trackers.

Features used in this experiment:

1. timestamp – Date and time at which the activity was recorded
2. steps – Number of steps recorded
3. calories – Calories burned
4. data source – Source of data collection (not used in clustering)

After preprocessing, the data was aggregated on a daily basis:

- Two numerical features:
  - Daily total steps
  - Daily total calories burned

3. Mathematical Formulation of the Algorithm

An Artificial Neural Network (ANN) consists of layers of neurons where each neuron performs a weighted sum followed by an activation function.

Forward Propagation:

For a neuron:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$a = f(z)$$

Where:

x_i = input features
w_i = weights
b = bias
z = linear combination
a = activation output
$f(z)$ = activation function (ReLU / Softmax)

For Hidden Layers (ReLU activation):

$$f(z) = \max(0, z)$$

For Output Layer (Softmax for multi-class classification):

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Loss Function (Categorical Cross-Entropy):

$$L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

Where:

- y_i = actual label
- $\hat{y}_i$ = predicted probability

Backpropagation updates weights using gradient descent:

$$w = w - \eta \frac{\partial L}{\partial w}$$

Where:

- η= learning rate

4.Algorithm Limitations

- Requires large dataset for good performance
- Prone to overfitting if not regularized
- Computationally expensive
- Requires hyperparameter tuning
- Acts as a black-box model (low interpretability)
- Sensitive to feature scaling

5.Methodology/Workflow

Step 1: Load multiple CSV files containing steps and calories data
Step 2: Clean data and remove unnecessary columns
Step 3: Convert timestamp to date format
Step 4: Aggregate daily steps and calories
Step 5: Create activity labels (Low, Moderate, High)
Step 6: Encode labels into numerical form
Step 7: Normalize features using scaling
Step 8: Split dataset into training and testing sets
Step 9: Build ANN model using Dense layers
Step 10: Train the model using forward and backpropagation
Step 11: Evaluate model performance using test data
Step 12: Visualize accuracy and loss graphs

Workflow Representation:

Raw Data
→ Data Cleaning
→ Feature Engineering
→ Label Creation
→ Feature Scaling
→ Train-Test Split
→ ANN Model Training
→ Evaluation
→ Visualization

6.Performance Analysis

The model performance was evaluated using the following metrics:

Accuracy:

Accuracy=Correct PredictionsTotal PredictionsAccuracy = \frac{Correct\ Predictions}{Total\ Predictions}Accuracy=Total PredictionsCorrect Predictions

Confusion Matrix:

Used to analyze classification performance across all classes.

Classification Report:

Includes:

- Precision
- Recall
- F1-score

Observation:

- The model was able to classify activity levels effectively
- Accuracy improved over epochs
- Loss decreased, indicating proper learning
- Validation accuracy helped monitor overfitting

7.Hyperparameter tuning

The following hyperparameters were tuned:

- Number of hidden layers
- Number of neurons in each layer
- Learning rate (Adam optimizer)
- Number of epochs
- Batch size
- Activation functions

Tuning Approach:

- Different network architectures were tested
- Epochs were adjusted to balance learning and overfitting
- Batch size was varied for stable training
- ReLU was used for hidden layers and Softmax for output

Impact:

- Proper tuning improved model accuracy
- Reduced overfitting and improved generalization
- Stabilized training process

Code:

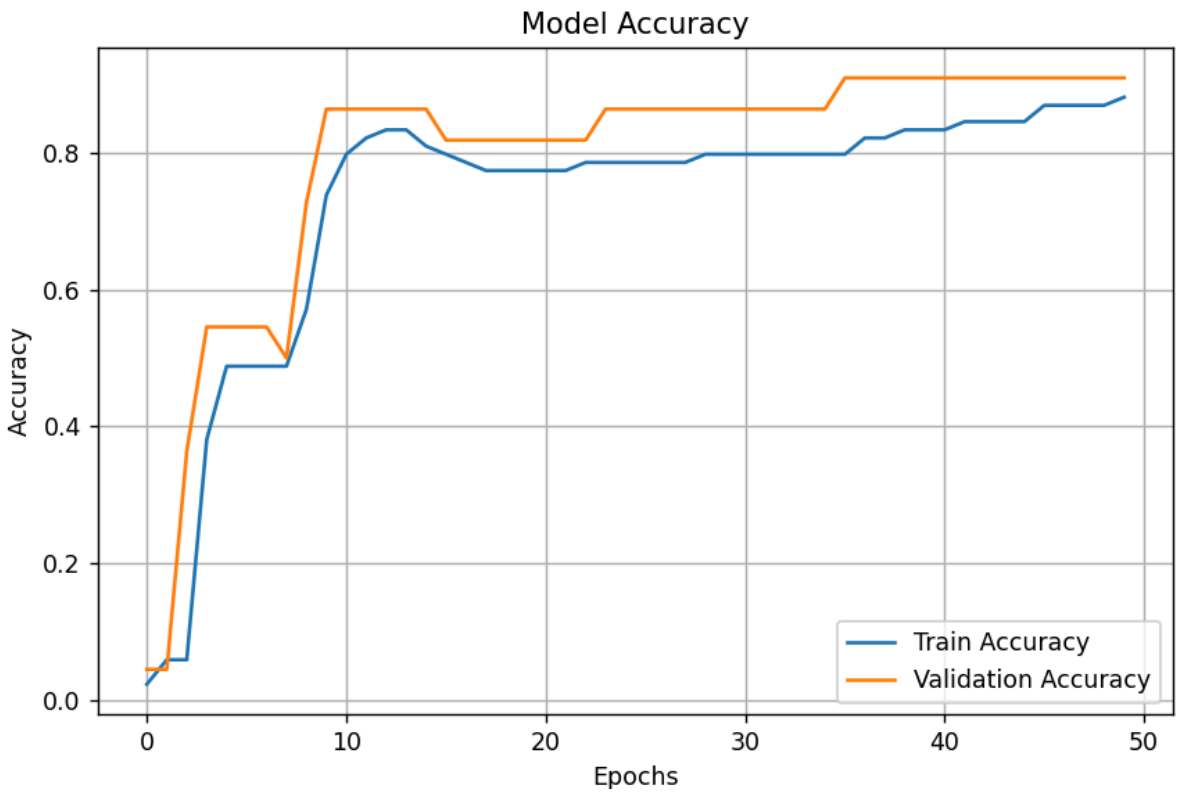
```python
import pandas as pd
import glob
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense


step_files = glob.glob("steps_data/*.csv")
cal_files = glob.glob("calories_data/*.csv")

steps_df = pd.concat([pd.read_csv(f) for f in step_files], ignore_index=True)
cal_df = pd.concat([pd.read_csv(f) for f in cal_files], ignore_index=True)


steps_df = steps_df[['timestamp', 'steps']]
cal_df = cal_df[['timestamp', 'calories']]

steps_df['timestamp'] = pd.to_datetime(steps_df['timestamp'])
cal_df['timestamp'] = pd.to_datetime(cal_df['timestamp'])

steps_df['date'] = steps_df['timestamp'].dt.date
cal_df['date'] = cal_df['timestamp'].dt.date

daily_steps = steps_df.groupby('date')['steps'].sum().reset_index()
daily_cal = cal_df.groupby('date')['calories'].sum().reset_index()

daily = pd.merge(daily_steps, daily_cal, on='date')

q1 = daily['steps'].quantile(0.33)
q2 = daily['steps'].quantile(0.66)

```

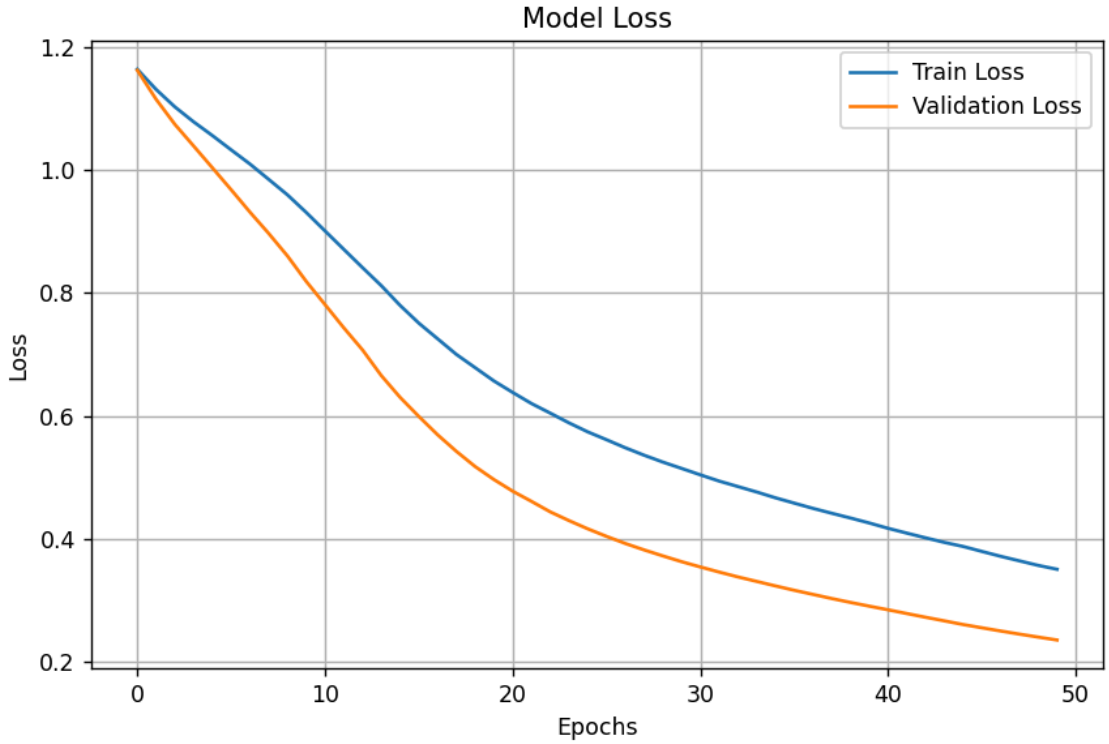
```python
def label_activity(x):
    if x <= q1:
        return "Low"
    elif x <= q2:
        return "Moderate"
    else:
        return "High"

daily['Activity'] = daily['steps'].apply(label_activity)

print(daily['Activity'].value_counts())

X = daily[['steps', 'calories']].values
y = daily['Activity'].values


le = LabelEncoder()
y_encoded = le.fit_transform(y)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42
)


model = Sequential()
model.add(Dense(8, activation='relu', input_dim=2))
model.add(Dense(6, activation='relu'))
model.add(Dense(3, activation='softmax'))


model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
```

```python
)
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.2,
    verbose=1
)

plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Train Accuracy", "Validation Accuracy"])
plt.grid(True)
plt.show()

plt.figure(figsize=(8,5))
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Train Loss", "Validation Loss"])
plt.grid(True)
plt.show()
```

Output:

```
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7738 - loss: 0.6785 - val_accuracy: 0.8182 - val_loss: 0.5175
Epoch 20/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7738 - loss: 0.6569 - val_accuracy: 0.8182 - val_loss: 0.4963
Epoch 21/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7738 - loss: 0.6381 - val_accuracy: 0.8182 - val_loss: 0.4774
Epoch 22/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7738 - loss: 0.6201 - val_accuracy: 0.8182 - val_loss: 0.4608
Epoch 23/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7857 - loss: 0.6045 - val_accuracy: 0.8182 - val_loss: 0.4437
Epoch 24/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7857 - loss: 0.5886 - val_accuracy: 0.8636 - val_loss: 0.4294
Epoch 25/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7857 - loss: 0.5739 - val_accuracy: 0.8636 - val_loss: 0.4161
Epoch 26/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7857 - loss: 0.5612 - val_accuracy: 0.8636 - val_loss: 0.4038
Epoch 27/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7857 - loss: 0.5481 - val_accuracy: 0.8636 - val_loss: 0.3922
Epoch 28/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7857 - loss: 0.5360 - val_accuracy: 0.8636 - val_loss: 0.3818
Epoch 29/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7976 - loss: 0.5247 - val_accuracy: 0.8636 - val_loss: 0.3720
Epoch 30/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7976 - loss: 0.5144 - val_accuracy: 0.8636 - val_loss: 0.3626
Epoch 31/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7976 - loss: 0.5040 - val_accuracy: 0.8636 - val_loss: 0.3540
```






```
accuracy: 0.8810 - loss: 0.3502
```

Conclusion:

Artificial Neural Networks are powerful machine learning models capable of learning complex and non-linear relationships in data through layered structures and activation functions. By adjusting weights using forward propagation and backpropagation, ANNs can effectively perform classification and prediction tasks across various domains.

Their performance depends on factors such as data quality, feature scaling, network architecture, and hyperparameter tuning. When properly designed and trained, ANNs provide high accuracy and adaptability, making them suitable for handling real-world problems involving large and complex datasets.

However, they require careful tuning and sufficient data to avoid overfitting and ensure good generalization. Overall, Artificial Neural Networks serve as a robust and flexible approach in modern machine learning.